

Sortowanie przez wstawianie

„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Grzegorz Szymkowiak

Opis algorytmu

- Sortowanie przez wstawianie (insertion sort) polega na tym, że kolejne elementy tablicy „wstawia się” w odpowiednie miejsce w już uporządkowanej części danych



Metoda

Metoda

- Pierwszy element uznajemy za posortowany
- Bierzemy kolejny element - tzw. klucz
- Przesuwamy wszystkie większe elementy w prawo, tak aby zrobić dla niego miejsce
- Wstawiamy klucz w odpowiednie miejsce w uporządkowanej części tablicy
- Powtarzamy proces dla wszystkich elementów od indeksu 1 do $n-1$

Metoda

6 5 3 1 8 7 2 4

Algorytm

```
dla  $i \leftarrow 1, 2, \dots, n - 1$  wykonuj  
  pom  $\leftarrow A[i]$   
   $j \leftarrow i - 1$   
  dopóki  $j \geq 0$  oraz  $A[j] > \text{pom}$  wykonuj  
     $A[j+1] \leftarrow A[j]$   
     $j \leftarrow j - 1$   
   $A[j+1] \leftarrow \text{pom}$ 
```

Specyfikacja

Specyfikacja

Dane:

- Liczba naturalna $n > 0$ (liczba elementów tablicy)
- n -elementowa tablica jednowymiarowa zawierająca liczby rzeczywiste $T[0 \dots n-1]$

Wynik:

- Posortowana niemalejąco n -elementowa tablica jednowymiarowa zawierająca liczby rzeczywiste

Implementacja

Implementacja

```
void insertionSort(int A[], int n)
{
    for (int i = 1; i <= n - 1; i++)
    {
        int temp = A[i];
        int j = i - 1;

        while (j >= 0 && A[j] > temp)
        {
            A[j + 1] = A[j];
            j = j - 1;
        }

        A[j + 1] = temp;
    }
}
```

Implementacja

```
int main()
{
    int A[] = { 5, 2, 4, 6, 1, 3 };
    int n = sizeof(A) / sizeof(A[0]);

    insertionSort(A, n);

    for (int i = 0; i < n; i++)
        std::cout << A[i] << " ";
    std::cout << "\n";

    return 0;
}
```

Złożoność czasowa

Złożoność czasowa

- Algorytm sortowania przez wstawianie jest skuteczny dla małych zbiorów danych, ale staje się mniej efektywny dla większych zbiorów w porównaniu do bardziej zaawansowanych algorytmów sortowania

Złożoność czasowa

- Operacją dominującą w algorytmie jest porównanie - w tym przypadku dane wejściowe mają wpływ na złożoność algorytmu

Złożoność czasowa

- Wyznaczamy złożoność pesymistyczną, czyli największą możliwą liczbę porównań - taka sytuacja jest możliwa w ciągu uporządkowanym odwrotnie

Złożoność $O(n^2)$

Złożoność czasowa

- W przypadku sortowania ciągu prawie uporządkowanego mamy do czynienia z złożonością liniową $O(n)$

www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak